

Requirement Analysis - Team 4

By Abdul Razik Khairkhva, Adam Foster, Adam John, Ahmad Azizi, Aron Bachrati, Jan Rustom and Yan Kin Cheng

We are building a **Version Control and Productivity Web App** to help programmers manage large, potentially multi-person projects. It will include productivity tools to help users focus on progress rather than privacy concerns or unnecessary features. A clean, minimalist UI helps reduce distractions so users are able to log in, see any tickets that need completing, and begin work - with no extra hassle.

Project Description

Any modern, large software project requires some form of version control. This may be a very basic system such as keeping timestamped backups, but for more complex software developed by larger teams, a solution is needed to allow features such as working in parallel or fixing bugs across multiple versions. The most popular system for this is Git.

We propose a lightweight web-based Git hosting platform, similar in principle to GitHub, with integrated productivity tracking and peer validation. Developers will be able to start dedicated focus sessions, tied to the repository and view this in a customisable graph. They will also be able to peer-review code, helping to increase code quality. Repositories can also be managed remotely from the web. Our software will use Git's API as a basis for the version control, and build off it by hosting the repositories and adding collaboration and productivity features.

Aims and Objectives

Core Features

These are the essential features for a minimum viable product.

Aim: Create a web-based version control system using Git.

Users can:

- List individual commits for a selected repository and branch.
- Create, read, update, and delete files
- Make new commits in the browser.
- View all branches in a repository.
- Preview file content, then parse and render them as markdown (if appropriate).
- Preview diffs (changes) in each commit.
- Preview the repository, from a particular commit (checkout).
- Preview the repository, from a particular branch (checkout).

Aim: Integrate a productivity tracker.

Users can:

- Start focus sessions, tied to the repository.
- Save focus sessions before they are scheduled to finish.
- View focus data.
- Customise their repository's productivity graph.
- Rate Git commits with peer-review tags.
- Comment on Git commits.

Aim: Implement a frontend for user authentication, profiles, and settings.

Users can:

- Change the image avatar, associated with their account, in settings.
- Change their password, by selecting "Forgot Password", when logging in.
- See repositories that they work on

Bonus Features

These features are optional and not necessary for a minimum viable product, but would improve the product overall if included.

Aim: Allow collaborative repository ticket management.

Users can:

- Create, edit, and delete tickets.
- Mark tickets as "Resolved".
- Assign users to tickets.
- Create tags, with an optional associated colour.
- Add and remove tags from tickets.
- Add tickets to a category (Kanban).
- Switch between "Kanban" and "Tabular" views.
- Filter tickets by tags.

Aim: Include messaging for project-based discussions.

Users can:

- Create, edit, and delete discussion threads.
- Lock and unlock discussion threads.

Background and Literature

Modern version control platforms provide little measure of developer productivity. There has historically been a focus on version control record outputs (commits), but not inputs - the progress that happens between commits.

Amit et al. (2021) argue that combining Pomodoro, gamification and peer influence can significantly reduce procrastination. Platforms like trydeepwork.com attempt this approach, but most are individualised and not team-based applications. We address this with our productivity data - mimicking a competitive “score”.

Kotte et al. (2013) highlight that collaborative, context-aware evaluation processes improve the accuracy of software-quality assessments. We used this research to inform our focus on productivity via increasing the speed of code production and improving peer assessment relative to other similar products. Visualising time-based activity data has also been shown to improve learner reflection (Manso-Vázquez et al, 2016), so we included graphing of our productivity data.

Most of the technical API documentation for this project can be found at <https://git-scm.com/cheat-sheet>, and the rest can be found at <https://git-scm.com/docs>.

Existing products treat repository mining, productivity tracking, peer review, and activity visualisation as separate domains, while our software aims to bring them together. We view productivity as an intersection between work process, team perception, and resulting code changes as opposed to repository activity alone.

Development and Implementation Summary

We are building a website that uses client-side processing, to help with backend complexity and scaling. The frontend will include graphics and analytics, with the backend being mostly reserved for hosting the Git interface.

We plan to use library code for authentication.

Main components

Git UI

Allowing the users to commit and view code using the Git API. It should provide all the basic functionality expected of version control software, such as diff viewing and branch explorers.

Productivity

Giving users access to productivity data such as time spent, commits or issues flagged. Is unobtrusive into the main Git functionality. This data is displayed in tables, charts or graphs.

Ticket Tracker

Using a tag system, users can log issues and tickets, integrated with the Kanban board. Issues are a subclass of tickets, but can be filtered out by the user.

Technical Summary

The frontend will be written using React (JavaScript) to create the UI, chosen for its simplicity and extensive documentation and resources available online.

The backend will deal with the Git CLI, the MySQL database and the authentication. The backend will be written in Rust, chosen for its speed and efficiency.

Our project has been organised to allow a high parallelism of tasks - the frontend is divided into a section of individual tasks which can be worked on in small sub-teams in Agile-like sprints. The largest component, being the Git UI, will likely span most of the project development, but smaller sections e.g. productivity can be done over a shorter timeframe. This organisation means that feature testing can be done earlier, meaning there is less to do when we are near the end of the project - leaving mostly beta and integration testing.

We have picked an ambitious, feature heavy, scope to ensure there is enough work for our team to do. The features we've chosen have been nicely split into priority levels, meaning that we have a focus on the core features but can expand into the optional features if the project is progressing faster.

Data Sources and Ethical Considerations

Ethical considerations primarily around user privacy, consent, and the responsible use of productivity data. Features such as commit sessions, time tracking, shared productivity goals, and peer validation involve collecting and displaying data about individuals' working patterns, effort, and perceived performance.

We have read over the university's ethical guidelines (University of Liverpool, n.d.) and will follow them with this project.

Data Protection

The data processed by this project includes source code, commit histories, documentation, tickets, comments, repositories and some basic account information (e.g., username and email address) provided by users. This data will be stored in our database and Git repository, but no processing will be done. All data processing will be done locally on the user's frontend.

All data uploaded or generated by users of our platform will be stored and kept secure in accordance with UK GDPR and the Data Protection Act (2018). We will only store data that is necessary for the platform to run, and delete user's data if requested. One of the perks of our project compared to existing products is our focus on privacy.

Testing

We would like to use beta testers as part of our Testing and Evaluation plan. We will ensure that any participants are adults from the UK, with no incentive to participate. They will be fully aware of what they are testing and their role in development.

We will test our project using sample code from open source repositories online, ensuring that we only use code that is public domain or with a license that allows us to use it.

Communication

Any features that allow communication in our software, for example comments or forums, should be managed by the user who owns that project. We will not be implementing any moderation since this breaches our privacy commitment, instead the users will have control over how widely their projects are visible and interacted with, including the ability to block others from viewing or commenting on their projects.

To reduce the risk of unauthenticated or unauthorized access, sensitive data (such as passwords) will be encrypted before being stored, and secure login mechanisms will be used. Access control restrictions will also be enforced to ensure that private repositories are only visible to authorized users.

Testing

Verification:

Our background research was conducted to ensure that the product satisfies our Aims.

Unit testing: After each (frontend) module is completed, unit testing is done to ensure that there are no bugs before adding it to the main project.

Integration testing: Integration testing integrates APIs with backend data. Replace stubs and drivers with real logic, data. Connections between frontend and backend are evaluated and identified defects are fixed.

Top-down testing: Top-down testing with stubs (sample file, default response) is built. High-level features (navigation, page transitions) are evaluated and tested to check whether APIs work properly.

Bottom-up testing: Once backend logic is done, bottom-up testing is created to test whether backend logic and database operation is correct.

Black-box testing: Black-box testing is performed after functions on the web are decided. Multiple test cases are executed to ensure functions work as expected and meet user requirements.

White-box testing: White-box testing is set up after the backend is completed to test every path of code.

Performance and stressing testing: Under performance and stressing testing, response time, reliability and stability are measured under different loads. The measurements are evaluated whether they meet expectations or not.

Validation:

Beta-testing: Beta-testing is conducted with various users to ensure the web meets users' expectations. Feedback is collected through questionnaires that assess accessibility, usability and suggested improvements.

BCS Compliance

We have chosen a particularly innovative project - our focus on bringing a simplicity and productivity spin to a standard version control project.

The core aim of our project is productivity, to give users a service that is streamlined, low overhead and with no excess features - for people who just want to get stuff done. This provides an application for programmers who are sick of the privacy concerns or feature overload of services like GitHub.

Our plan has a strong focus on testing, to ensure both validation and verification of the final product. Furthermore, we have a lot of well-defined criteria and requirements to inform these tests to evaluate the "quality" of our final product.

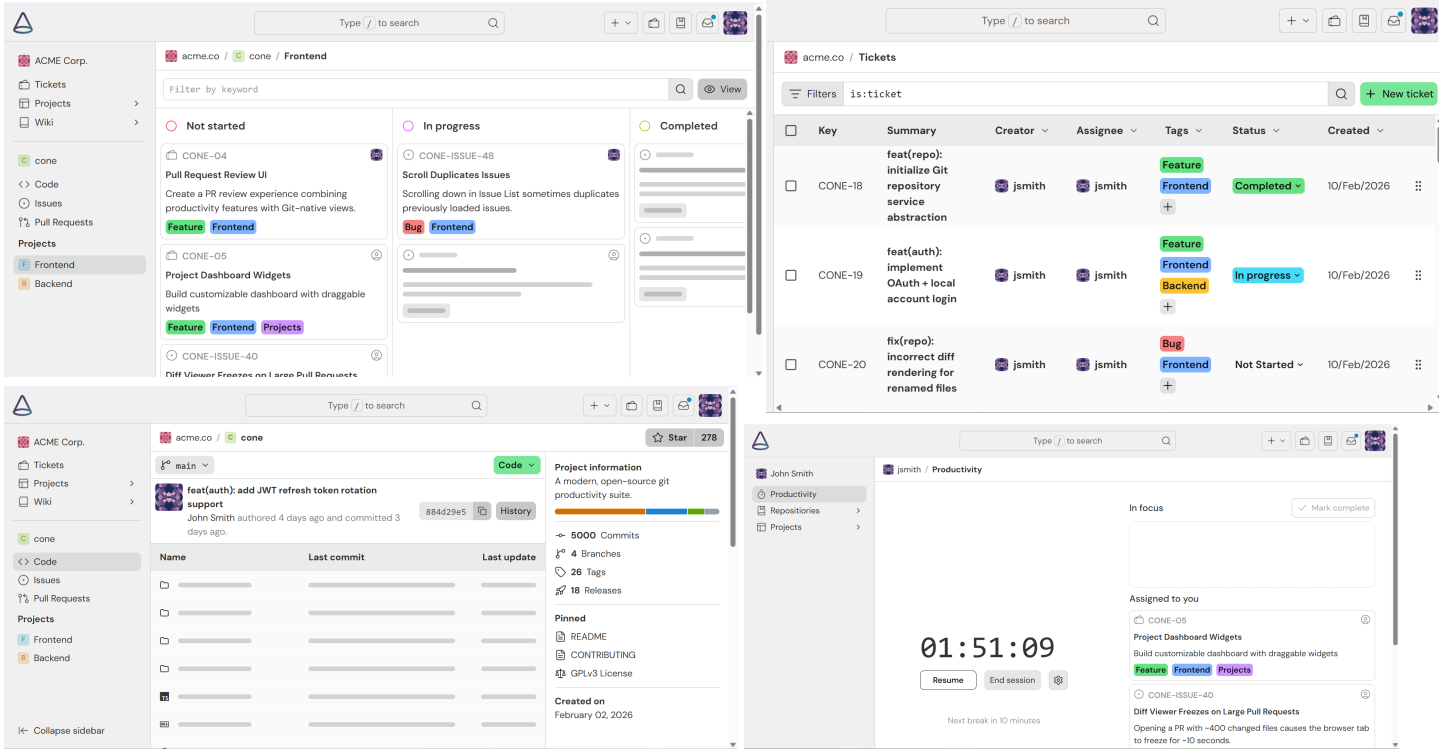
We have regular meetings with clear agendas and documentation. We have set up a casual but organised and well-structured Discord server for communication and planning. Each meeting ends with the next being arranged, so that everyone always has a clear task and deadline.

Whilst we are following a similar ideology to Agile with smaller teams working in "sprints", we are not strictly confined to any rigid schedule. We are looking at making useful prototypes early on to test features, allowing us to assign tasks more dynamically based on the issues we face. Evaluation will form a large part of our final submission report, as well as every meeting having time allocated to check and assess each other's work.

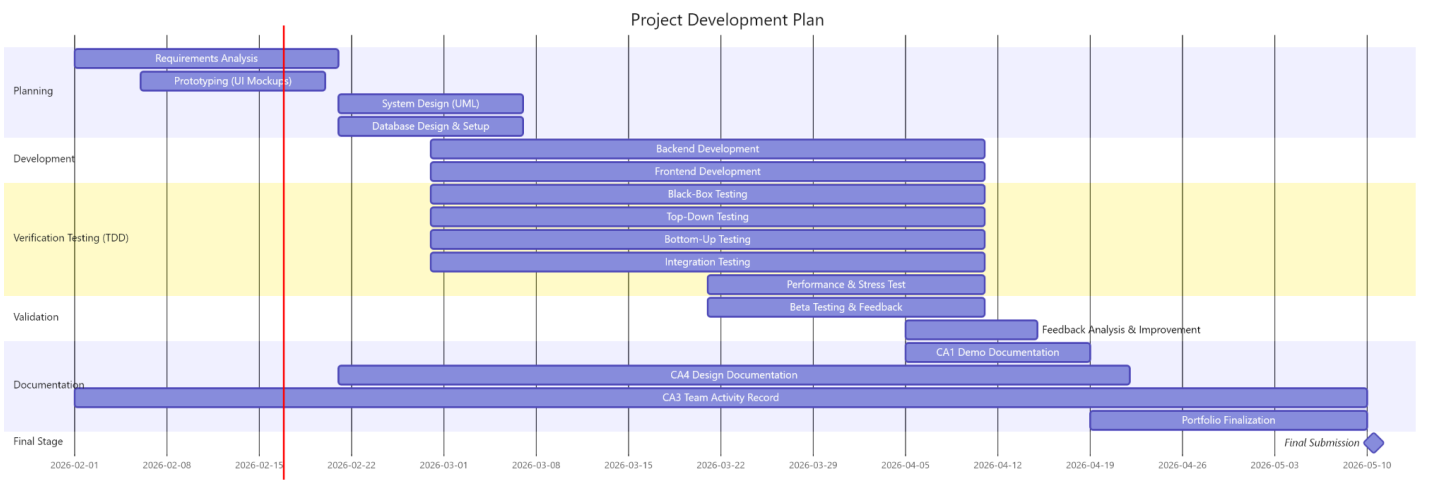
Creating this project plan has drawn on many skills from the degree: most notably Designing Systems For The Digital Society (COMP107) and Software Engineering I (COMP201) for their project management considerations and planning skills. As we progress further into the technical sections, we will start using more of the skills from Object-Oriented Programming (COMP122) and Database Development (COMP207).

We have read the BCS Accreditation Guidelines (BCS, 2022), and our project follows this.

UI Mockup



Planning, Risk and Contingency Plan



Risk	Contingencies	Likelihood	Impact
Hardware failure (laptop crash)	Backup data in cloud/ commit code regularly	Medium	High
Software failure (IDE/ framework)	Use stable version/ reinstall if necessary	Medium	Medium
Programming errors/ Logic errors	Test regularly, debugging, peer code viewing	High	Medium
Team member unavailable (sick/ accident)	Share work document regularly and task tracking	Low	Medium
Database errors/ corruption (concurrency issues/ deadlock)	Use backup, test queries regularly	Low	Very high

Risk	Contingencies	Likelihood	Impact
Hardware failure (laptop crash)	Backup data in cloud/ commit code regularly	Medium	High
Software failure (IDE/ framework)	Use stable version/ reinstall if necessary	Medium	Medium
Programming errors/ Logic errors	Test regularly, debugging, peer code viewing	High	Medium
Underestimate develop time needed	Set a buffer time, prioritize core feature/ drop optional feature if needed	High	Very high
Integration issue (API mismatch)	Design before creating (UML diagram) and add to documentation	Medium	High
Cross-platform compatibility issue (browser/ software)	Conduct cross-platform testing (Chrome/ Safari), responsive UI testing	Medium	Low
Third-party service down (OAuth2)	Test with stubs/ mock APIs	Low	Medium
Project file lost	Use backup service (git-hub) enable version control	Low	High

BIBLIOGRAPHY

Amit, A.J., Gautam Shankararam, S.R., Pradeep, P., Perumalraja, R. and Kamalesh, S. (2021). Framework For Preventing Procrastination And Increasing Productivity. IEEE doi:10.1109/ICSPC51351.2021.9451773. (Accessed: 11th February 2026).

BCS (2022) Accreditation Guidelines. Available at: <https://www.bcs.org/media/1209/accreditation-guidelines.pdf> (Accessed: 9th February 2026).

Kotte, O., Elorriaga, A., Stokic, D. and Scholze, S. (2013). Context Sensitive Solution for Collaborative Decision Making on Quality Assurance in Software Development Processes. Frontiers in Artificial Intelligence and Applications. doi:10.3233/978-1-61499-264-6-130. (Accessed: 11th February 2026).

Manso-Vázquez, M., Caeiro-Rodríguez, M. and Llamas-Nistal, M. (2016). Tracking and visualizing time management for Self-Regulated Learners. IEEE Frontiers in Education Conference (FIE). doi:10.1109/FIE.2016.7757411. (Accessed: 11th February 2026).

Software Freedom Conservancy. (n.d.) Git Documentation. Available at: <https://git-scm.com/> (Accessed: 11th February 2026).

University of Liverpool. (n.d.) Research ethics policy | Research | University of Liverpool. Available at: <https://www.liverpool.ac.uk/research/research-environment/ethics/policies-and-guidance/ethics-policy/> (Accessed: 15th February 2026).

VK. trydeepwork. (n.d.) What did you get done this week? Available at: <https://trydeepwork.com> (Accessed: 11th February 2026).